

Respuestas de la lección uno:

1._E y D

La estructura E sigue las reglas convencionales para que sea un método main, y la D aunque no es común ver el 'final' no cambia en nada la función del método, ya que al igual que el static no puede ser sobrescrito.

2._C

Es el orden correcto, esto es porque primero se define la ubicación del archivo(package), después lo que se va a usar(import) y ya después se escribe la clase.

3._A y E

Porque Bunny sí es una clase y bun es una referencia a un objeto. Las demás no aplican.

4._B,E y G

Son las únicas opciones que si cumplen con las reglas para nombrar, por ejemplo no comienzan con números, no llevan puntos o un signo no valido, etc.

5._A,D y F

Porque en la línea 9 y 10 ambos objetos tienen referencia, en la línea 11 el objeto de brownBear apunta ahora al objeto de polarBear, por ende el objeto de polarBear ahora tiene 2 líneas de referencia, mientras que el objeto de brownBear sigue teniendo solo una referencia, ya en la línea 12 polarBear ya no apunta ningún objeto, sin embargo el objeto no desaparece por que browBear sigue apuntando a ese objeto, ya en la 13 todas las referencias desaparecen y los objetos que dan al aire, por lo tanto en la línea 14 ya encontramos la llama al método que le sugiere a Java que ya se puede llevar esos objetos.

6._F

Fui quitando las variables que ya "murieron" porque salieron de su bloque. Al final solo conté las que siguen vivas en la línea 14 y son 7.

7._C y E

Esto por que todo dentro de 3 comillas se vera como solo texto, no interpreta código ni nada, solo texto y se imprime tal cual.

8._ B, D, E, H

Para utilizar el var debe haber un valor claro, un valor en el que Java pueda definir si es un int, string, boolean, etc.

9._E

Por que Java pone como default un null, no vacio, no 0, si no un null.

10._ A, E y F

En números no puedes usar los guiones bajos al inicio, tampoco al final y nunc pegado a un punto.

11._E

Hay 2 que viene del mismo paquete, no es necesario tenerlos, con la principal basta, en el caso de los .lang se integran automáticamente cuando el programa compila.

12._ A, C y D

En la línea 2 da error porque no se mezclan los tipos de variables, en la línea 4 java no permite que ya exista un valor determinado en los parámetros, porque estos se recibirán como argumentos, y finalmente 'fins' es una variable local que solo vive dentro del bloque.

13._ A, B y C

Los imports deben identificar qué clase Water se está usando. Si hay dos clases con el mismo nombre, Java no puede decidir, entonces no compila.

14._ A, B, D y E

En este caso short y long no son compatibles, in y double tampoco, en los otros dos casos los primitivos no tienen métodos y el .length solo existe en arreglos.

15._ C, E y F

Las demás son falsas por que no se le puede decir al gc cuando limpiar, en que momento exactamente, solo se le hace la sugerencia que ya puede hacerlo, y en el 'final' significa que la referencia no puede cambiar, pero el objeto si puede modificarse, y si queda libre entonces se lo puede llevar el gc.

16._A y D

Por que solo imprime 2 líneas, que son:

```
squirrel  
pigeon termite
```

y la primera línea tiene un espacio al final

17._D, F y G

El programa no da error porque en todas las variables Java le asigna un valor por defecto. El boolean se inicializa en false, el String en null y el float en 0.0.

18._B, C y F

'var' permite que Java asigne el tipo de valor automáticamente, pero ese tipo se define en compilación y no puede cambiar después. Además, solo se puede utilizar con variables locales.

19._A y D

Con `parseLong` se convierte el "100" a un número primitivo, mientras que con `valueOf` se convierte en un objeto. Para poder compararlos, Java automáticamente convierte el objeto a primitivo.

20._C

El método `PoliceBox` no es un constructor porque tiene `void`, así que nunca se ejecuta. Por eso los valores quedan como `null` y `0`. Aunque se cambian en `p`, pero después `p = q` hace que ambos apunten al mismo objeto con valores por defecto.

21._D

Como no hay `static` el programa comienza a ejecutarse de el `main`, ahí imprime el 7, luego se va a la clase `Salmon`, comienza con los bloques que hay, el primero es el de la línea 3 que da 0, luego el de la línea 4 queda 1, y por último el constructor que da 2, finalmente el ultimo `println` del `main` que da 4

22._C, F y G

La 'C' es correcta debido a que el numero es un hexadecimal que da 14 por lo tanto entra en el `int`, la 'F' da igual a 5 por ende también entra dentro del `int`, y finalmente la 'G' los guiones están en una posición correcta y es como decir 92.12 por lo tanto entra en el `double`, mientras que los demás intenta que un `int` acepte un `long`, o que los espacios están mal colocados, etc.

23._A y D

Da error en la línea 3 debido a que falto asignar el 'f' al final del número para decir que la variable es `float`, y luego en la línea 10 el `print` esta tratando de utilizar la variable 'depth', pero este solo es local para el `for`, fuera de este `for` no existe.